MODULE 5 · STORAGE

PVCs in Deployments & StatefulSets

One shared claim, or one per replica — and why that choice breaks at scale

WHY IT MATTERS

Two workload types wire the same PVC field differently

Deployments and StatefulSets both mount PersistentVolumeClaims, but they wire them up in opposite ways. A Deployment's Pod template names **one fixed PVC**, so every replica it stamps out shares it. A StatefulSet uses `volumeClaimTemplates` to mint a **dedicated PVC per replica**.

Pick the wrong shape and you either break scaling with a contended single-node volume, or pay for isolation a stateless app never needed.

BY THE END YOU CAN

- Explain why a Deployment + PVC fails past one node
- Read how `volumeClaimTemplates` names each PVC
- Verify one PVC per StatefulSet replica
- Choose shared vs per-pod storage correctly

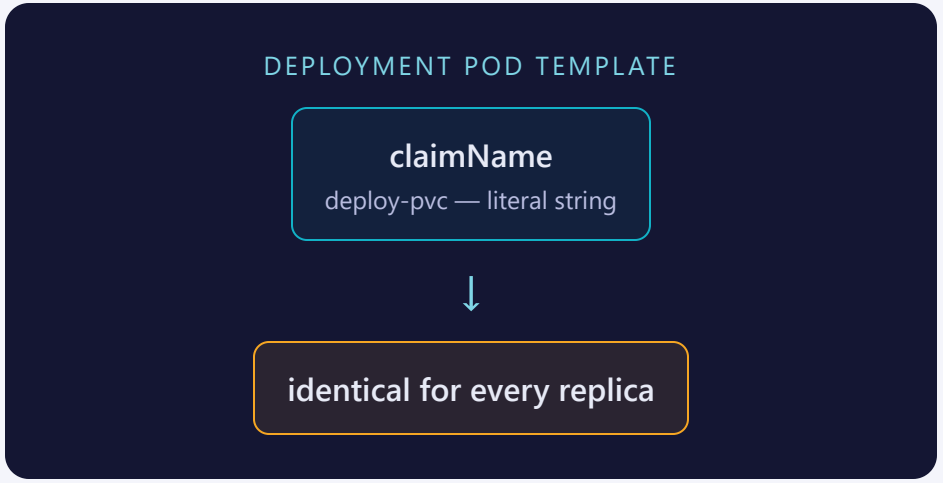
CONCEPT

One field decides whether Pods share storage

A Deployment's Pod template sets `spec.volumes[].persistentVolumeClaim.claimName` to one literal string. Every replica stamped from that template resolves to the exact same PVC.

A StatefulSet has no `claimName` at all. Its `volumeClaimTemplates` is a PVC template — the controller creates a real, uniquely-named PVC for every ordinal and binds each Pod to its own.

- Deployment Pods are interchangeable clones → one shared claim is fine for stateless data
- StatefulSet Pods keep a stable identity → each needs its own stable claim to match



TOPIC DEEP-DIVE

Shared claim vs one claim per replica

MANIFEST ANATOMY

A template that stamps out one PVC per ordinal

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web-stateful
  namespace: training
spec:
  serviceName: web-headless   # must match the headless Svc name
  replicas: 3
  selector:
    matchLabels:
      app: web-stateful
  template:
    metadata:
      labels:
        app: web-stateful
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
              name: web
          volumeMounts:
            - name: web-data
              mountPath: /usr/share/nginx/html
  volumeClaimTemplates:          # stamped once per ordinal
    - metadata:
        name: web-data
        labels:
          app: web-stateful
      spec:
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 100Mi
```

volumeClaimTemplates sits beside `template`, at the StatefulSet's own `spec` — the controller mints a real PVC per ordinal, named `<template-name>-<statefulset>-<ordinal>`, e.g. `web-data-web-stateful-0`, `-1`, `-2`.

MANIFEST ANATOMY — THE CONTRAST

The PVC is created once, up front

```yaml
# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: deploy-pvc
  namespace: training
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 200Mi
```

A plain, standalone PVC — no template, no owner, just a name other objects reference by `claimName`.

One literal claimName, no template at all

```yaml
# deployment-pvc.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deploy-with-pvc
  namespace: training
  labels:
    app: deploy-pvc
spec:
  replicas: 1
  selector:
    matchLabels:
      app: deploy-pvc
  template:
    metadata:
      labels:
        app: deploy-pvc
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          volumeMounts:
            - name: web-data
              mountPath: /usr/share/nginx/html
      volumes:
        - name: web-data
          persistentVolumeClaim:
            claimName: deploy-pvc
```

Every Pod template that sets `claimName: deploy-pvc` — no matter how many replicas — resolves to that same object; there's no template to mint a new one per replica.

DO IT — TWO WAYS

`kubectl create` builds the workload — storage still needs hand-editing

IMPERATIVE — A STARTING POINT	DECLARATIVE — THE ONLY WAY TO WIRE STORAGE
`kubectl create deployment deploy-with-pvc --image=nginx:1.25`	`kubectl apply -f pvc.yaml -f deployment-pvc.yaml`
Gives you a bare Deployment — no volumes, no claim	Hand-write `spec.volumes` or `volumeClaimTemplates` yourself

THE BRIDGE TRICK

Generate the workload skeleton, then hand-edit in the volume — no imperative flag adds one:

```
$ kubectl create deployment deploy-with-pvc --image=nginx:1.25 \
    --dry-run=client -o yaml > deployment-pvc.yaml  # then add spec.volumes by hand
```

SEE IT

The PVC list is the proof

```
$ kubectl get pvc -n training
NAME                       STATUS   CAPACITY   ACCESS MODES
deploy-pvc                 Bound    200Mi      RWO
web-data-web-stateful-0    Bound    100Mi      RWO
web-data-web-stateful-1    Bound    100Mi      RWO
web-data-web-stateful-2    Bound    100Mi      RWO
```

One PVC total, no matter how many Deployment replicas share it. Exactly one PVC per StatefulSet ordinal, named after the Pod that owns it — the list itself shows the difference.

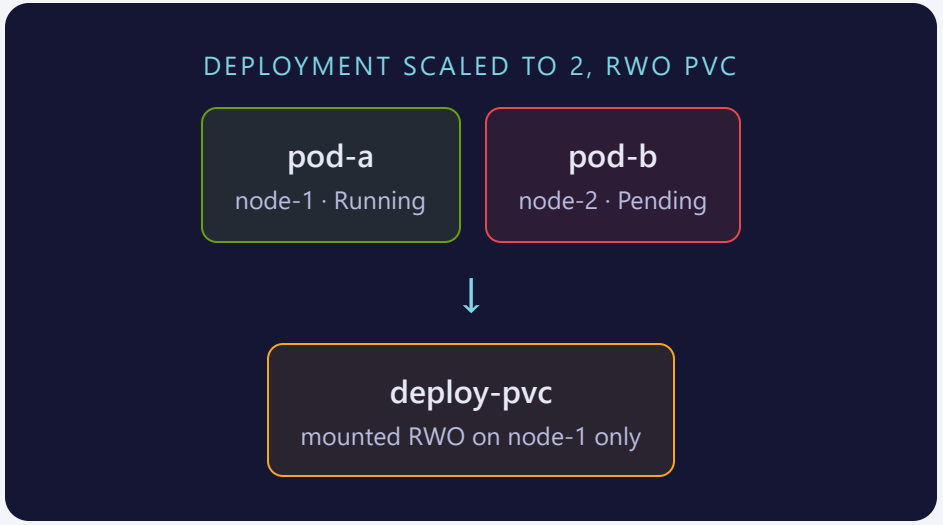
THE PUNCHLINE

Scale the Deployment past one replica, and RWO fights back

A `ReadWriteOnce` PVC can only be mounted read-write on one node at a time. Scale `deploy-with-pvc` from 1 to 2 and the new Pod schedules onto whichever node has room — often a different one.

If it lands on a different node, that Pod sticks in `Pending`: the volume is already mounted on the first node, and RWO forbids a second node from touching it.

A StatefulSet never hits this — each replica has its own PVC, so there's nothing to contend for.



GOTCHAS — THE ONES THAT BITE

Where people trip

- **Scaling a Deployment past 1 with an RWO PVC.** Works only if every replica happens to land on the same node — never guaranteed.
- **Assuming `volumeClaimTemplates` PVCs get deleted with the StatefulSet.** They don't — scale-down and deletion leave them behind on purpose.
- **Renaming a StatefulSet.** The PVC name embeds the StatefulSet's name — a rename orphans the old claims.
- **Treating a Deployment's PVC as per-replica storage.** It's one shared filesystem, not isolated storage per Pod.

YOUR PVC-IN-WORKLOADS TOOLKIT

Commands to keep at hand

```
$ kubectl get pvc -n NS                              # one row per claim — count them
$ kubectl get pvc -n NS -l app=LABEL                 # just a StatefulSet's own claims
$ kubectl describe pvc NAME -n NS                   # bound PV, access modes, capacity
$ kubectl scale deployment NAME --replicas=N -n NS # watch RWO break past one node
$ kubectl delete pvc -n NS -l app=LABEL              # StatefulSet PVCs need manual cleanup
```

Tip: `describe` a `Pending` Pod before blaming the image — a stuck RWO mount looks identical to a scheduling failure.

RECAP

PVCs in workloads in one breath

- A Deployment's Pod template names one fixed PVC — every replica shares it
- A StatefulSet's `volumeClaimTemplates` mints one PVC per ordinal, named `<template>-<sts>-<n>`
- RWO + a shared PVC breaks the moment replicas span more than one node
- StatefulSet PVCs outlive scale-down and deletion — clean them up yourself

HANDS-ON

Now run `Lab 5.4`

`labs/5.4/`

Next: **5.5 — Access Modes & Reclaim Policies**